

Overfitting, Regularization & Validation

KAUST–Mawhiba IOAI Training

Week 1 · Day 4



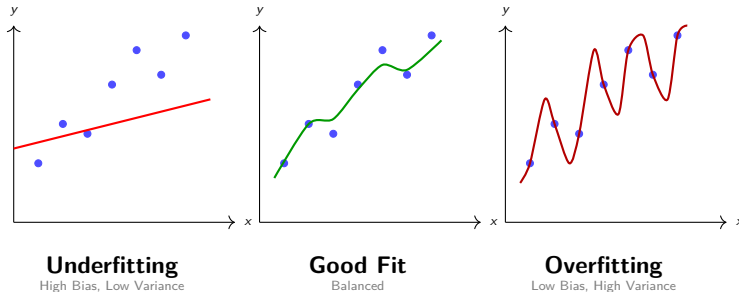
جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

April 25, 2026

1. Underfitting, Overfitting & Bias-Variance Tradeoff
2. Regularization: Ridge (L2) & Lasso (L1)
3. Cross-Validation: K-Fold, Stratified K-Fold, Group K-Fold
4. Hyperparameter Tuning: Grid & Random Search

Underfitting vs. Overfitting



Model is **too simple**.
Misses the pattern.

Model captures the
true relationship.

Model is **too complex**.
Memorizes noise.

Decomposition of Test Error

$$\text{Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Bias

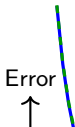
Error from wrong assumptions.
High bias $\rightarrow$ underfitting.

"How far is the average prediction from the truth?"

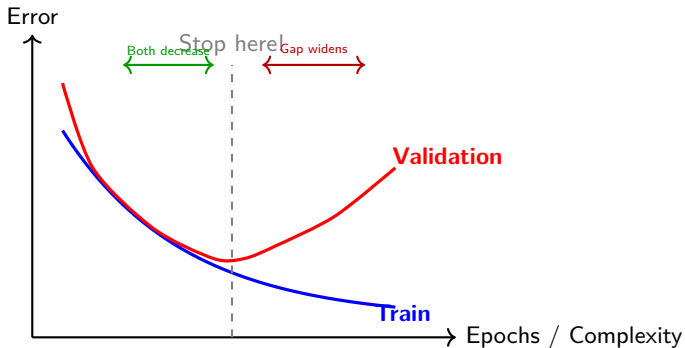
Variance

Error from sensitivity to training data.
High variance $\rightarrow$ overfitting.

"How much do predictions change across different training sets?"



How to Detect Overfitting



Key signal: Training error keeps decreasing, but validation error starts **increasing**. The growing gap = overfitting.

Imagine we train two linear models on the same data. Both achieve **Training MSE = 0.10**. Which one is better?

Model A (Small Weights)

$$y = 0.5x_1 + 0.3x_2$$

Val MSE: 0.12

Stable. Why?

→ Small changes in x cause
Small changes in y .

Model B (Huge Weights)

$$y = 500.5x_1 - 429.7x_2$$

Val MSE: 15.0 (!!!)

Unstable. Why?

→ Small changes in x cause
Massive changes in y .

Observation: Large weights are often a symptom of **Overfitting**. The model is trying too hard to wiggle through every training point.

The Fix: Constraining the Weights

We must force the model to keep weights small. We do this by adding a **Penalty** to the loss function.

$$\begin{array}{ccc} \text{Minimize Error} & + & \text{Minimize Weights} \\ \text{Fit the Data} & & \text{Keep Weights Small} \\ J_{final} = & \underbrace{J_{MSE}}_{\text{Fit the Data}} + & \underbrace{\lambda \cdot R(\mathbf{w})}_{\text{Keep Weights Small}} \end{array}$$

What is λ (Lambda)?

Regularization Strength (Hyperparameter):

- ▶ **High** λ : Huge penalty $\rightarrow$ Tiny weights (Model becomes simple) $\rightarrow$ Increases underfitting.
- ▶ **Low** λ : Small penalty $\rightarrow$ Weights can grow (Model fits data closely) $\rightarrow$ Increases overfitting.
- ▶ $\lambda = 0$: Standard Linear Regression (No penalty).

Let's set $\lambda = 1$ and Penalty $R(\mathbf{w}) = w^2$. Compare two models that have similar error, but different weights.

Model A (Overfit)

Weight $w = 100$

MSE Error = 0 (Perfect fit)

Total Loss:

$$J = 0 + 1 \cdot (100)^2$$

$$\mathbf{J} = 10,000$$

HUGE LOSS!

Model B (Balanced)

Weight $w = 2$

MSE Error = 5 (Some error)

Total Loss:

$$J = 5 + 1 \cdot (2)^2$$

$$\mathbf{J} = 9$$

WINNER!

We penalize the **Squared Magnitude** of the weights.

Objective Function

$$J(\mathbf{w}) = \text{MSE} + \lambda \sum w_j^2$$

Effect: “Weight Decay”

- ▶ Shrinks weights smoothly toward 0 (stability).
- ▶ Usually does not make weights exactly 0 (not sparse).
- ▶ Works well with correlated features.

Ridge Normal Equation

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

We penalize the **Absolute Value** of the weights.

Objective Function

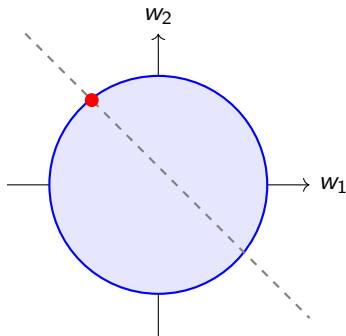
$$J(\mathbf{w}) = \text{MSE} + \lambda \sum |w_j|$$

Effect: “Sparsity”

- ▶ Encourages **sparsity**: many weights become exactly 0.
- ▶ Acts as **automatic feature selection**.
- ▶ Good when we believe **only a few features matter**.

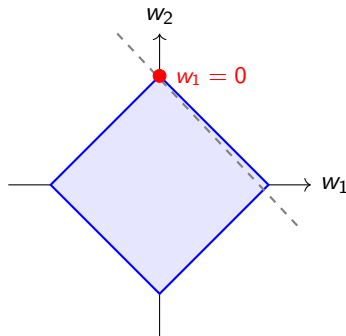
No closed-form solution (absolute value is not differentiable at 0). Must use **iterative optimization**.

L1 vs L2: Geometric Intuition



Loss Contour + L2 Constraint

Ridge: Smooth solution, rarely hits axis.
 $w_1 = -1.5, w_2 = 2.0$



Loss Contour + L1 Constraint

Lasso: Sharp corners promote sparsity.
 $w_1 = 0, w_2 = 2.5$

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda \sum w_j^2$	$\lambda \sum w_j $
Constraint Shape	Circle	Diamond
Sparsity	No (weights ≈ 0)	Yes (weights = 0)
Feature Selection	No	Yes
Closed-Form	Yes	No
Best When	Many useful features	Few important features

Elastic Net combines both: $J = \text{MSE} + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$

In Day 2 we learned **hold-out** splitting (80/20).

Problem: with small datasets, a single split may not be representative.

- ▶ Lucky split → optimistic score
- ▶ Unlucky split → pessimistic score

Solution: evaluate on *multiple* splits and **average the scores**.

Cross-validation also lets us **compare models** and **tune hyperparameters** more reliably.

- ▶ Split data into k equal parts (**folds**)
 - Train on $k-1$ folds, test on the remaining fold
 - Repeat k times, rotating the test fold
 - Average the k scores
- ▶ Every sample is in the test set **exactly once**
- ▶ Common choice: $k = 5$ or $k = 10$



- ▶ Same idea as K-Fold, but each fold preserves the **class distribution**
- ▶ Combines the benefits of K-Fold (robust evaluation) with stratification (balanced classes)

Code

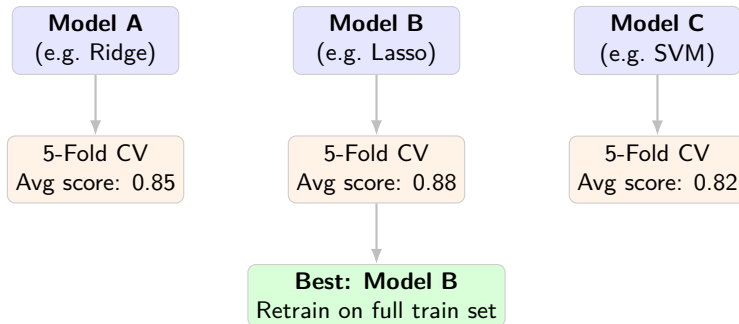
```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True)
for train_idx, test_idx in skf.split(X, y):
    X_train, X_test = X[train_idx], X[test_idx]
    # train model, compute score ...
```

- ▶ **Default choice** for classification — use this unless you have a reason not to

- ▶ Sometimes samples are **not independent** — they belong to groups
 - e.g. multiple medical images from the **same patient**
 - e.g. several transactions from the **same user**
- ▶ If the same patient appears in both train and test, the model can “cheat”
- ▶ **Group K-Fold** ensures all samples from one group are in the **same fold**

Code

```
from sklearn.model_selection import GroupKFold
gkf = GroupKFold(n_splits=5)
for train_idx, test_idx in gkf.split(X, y,
groups=patient_ids):
# no patient appears in both train and test
```

Use CV scores to **pick the best model / hyperparameters**, then retrain the winner on the **entire training set** before evaluating on the held-out test set.

Parameters (θ)

Learned from data during training.

- ▶ Weights $\mathbf{w}$ in linear regression
- ▶ Support vectors in SVM
- ▶ Split thresholds in decision trees

Set by the **optimizer**.

Hyperparameters

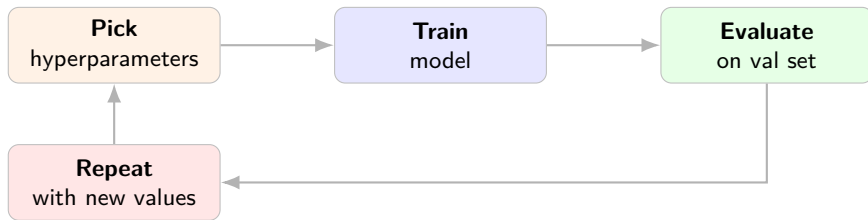
Set by us before training.

- ▶ Learning rate α
- ▶ Regularization λ
- ▶ Number of folds k
- ▶ Tree depth, # estimators

Set by **tuning / search**.

Key insight: Hyperparameters control model complexity. Choosing them well is critical to avoid overfitting / underfitting.

Goal: Find the hyperparameter combination that gives the best **validation** performance.



How do we choose which combinations to try?

1. **Grid Search** — try every combination
2. **Random Search** — sample randomly

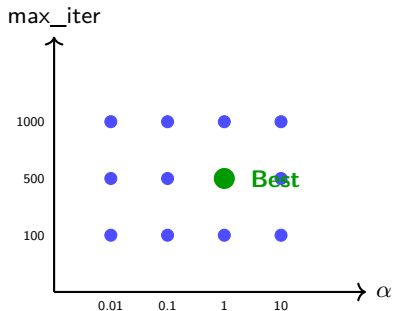
Idea: Define a grid of values for each hyperparameter and try **every combination**.

```
from sklearn.model_selection
import GridSearchCV

param_grid = {
    'alpha': [0.01, 0.1, 1, 10],
    'max_iter': [100, 500, 1000]
}

grid = GridSearchCV(
    model, param_grid, cv=5)
grid.fit(X_train, y_train)
```

$4 \times 3 = 12$ combinations $\times$ 5 folds = 60 fits



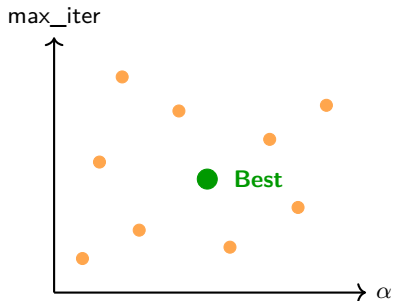
Exhaustive but **expensive**.
Cost grows exponentially with #
hyperparameters.

Idea: Sample hyperparameters **randomly** from distributions. Often finds good values faster.

```
from sklearn.model_selection
import RandomizedSearchCV

param_dist = {
    'alpha': uniform(0.001, 10),
    'max_iter': [100, 500, 1000]
}

search = RandomizedSearchCV(
    model, param_dist,
    n_iter=20, cv=5)
search.fit(X_train, y_train)
```



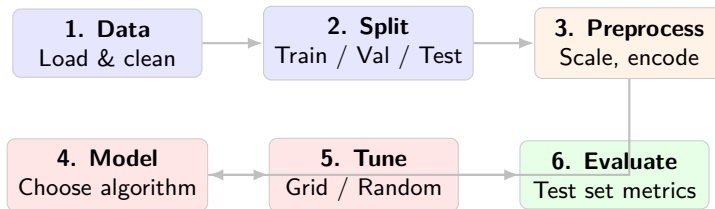
Explores **more of the space**.
Better when some hyperparameters
matter more than others.

	Grid Search	Random Search
Strategy	Exhaustive	Sampling
Coverage	Uniform grid points	Spread across space
Cost	$O(\prod n_i)$	$O(n_iter)$
Scales to many HPs	Poorly	Well
Guarantees optimum	In the grid, yes	No

Practical advice:

- ▶ Start with **Random Search** to find a good region.
- ▶ Then refine with **Grid Search** in that region.

Putting It All Together: The ML Pipeline



Golden rule: The **test set** is touched **only once** — at the very end. All tuning and model selection happens on **training + validation** data.

Concept	Key Takeaway
Overfitting	Model memorizes noise; high variance
Underfitting	Model too simple; high bias
Bias-Variance	Minimize total error at the sweet spot
Ridge (L2)	Shrinks weights smoothly; no sparsity
Lasso (L1)	Drives weights to zero; feature selection
Hyperparameters	Set before training; control complexity
Grid Search	Exhaustive; reliable but expensive
Random Search	Efficient; scales to many hyperparameters

Next: Classical ML Models — KNN, SVM, Decision Trees, Ensembles

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program